Section A – Architecture and Roadmap

---

1. Target End-to-End Architecture — Multi-Country Digital Direct Channel

Overview

The target architecture is designed for a multi-country insurance Digital Direct Channel, where Salesforce acts as the central system of record and MuleSoft Anypoint Platform as the universal integration middleware. Every inbound request from digital channels and every outbound request to external systems flows through the MuleSoft layer.

The architecture is built around four non-functional pillars:

- **High Availability:** No single point of failure; active-active deployment across multiple availability zones
- **Scalability:** Horizontal scaling at the MuleSoft runtime layer; Salesforce governor limits managed through bulkhead isolation
- **Resilience:** Retry with exponential backoff, circuit breaker, and bulkhead applied at the integration layer
- **Observability:** Unified logging, distributed traces (OpenTelemetry), and real-time dashboards across all integration flows

---

Architecture Layers

Layer 1 — Digital Channels (Consumers)

- Web portal (customer self-service)
- Mobile application
- Agent and broker portal
- External partners (APIs)

All channels communicate exclusively through MuleSoft Experience APIs over HTTPS/REST. No channel has direct access to Salesforce or backend systems.

Layer 2 — API-Led Integration with MuleSoft (Middleware)

Three-layer API-led connectivity model:

Experience APIs

- Channel-specific adapters: one per channel type (web, mobile, broker, partner)
- Handles authentication (OAuth 2.0 / Connected App), request validation, and response formatting
- Applies rate limiting and throttling per channel
- Injects correlation IDs and trace context (OpenTelemetry) into every request

Process APIs

- Orchestration layer: composes multiple System API calls to build business operations
- Implements business logic: quote orchestration, policy issuance, claims intake
- Applies resilience patterns: retry with exponential backoff + jitter, circuit breaker, idempotency key validation
- Routes long-running operations to the async messaging layer (per ADR-002)

- Manages error handling and fallback responses

System APIs

- Thin connectors to backend systems — no business logic
- Salesforce System API: exposes Salesforce objects (Account, Opportunity, Policy__c, Claim__c) as REST resources
- Payment Gateway System API
- Document Generation System API
- Per-country legacy system connectors

Layer 3 — Core Systems

- **Salesforce:** Insurance core (policies, claims, customers, products) — system of record
- **Payment Gateway:** Premium collection and refunds
- **Document Service:** Policies, certificates, claims reports
- **Notification Service:** Email, SMS, push notifications
- **Per-Country Systems:** Local regulatory systems, local payment providers

Layer 4 — Async Messaging Layer

For long-running and high-volume flows (per ADR-002):

- Message broker (Anypoint MQ or cloud-native equivalent)
- Dead Letter Queue (DLQ) for events that fail processing
- Event replay capability for incident recovery
- Per-country consumer isolation to contain processing failures

Layer 5 — Observability Platform

- **Distributed Traces:** OpenTelemetry SDK on MuleSoft runtimes; trace context propagated via W3C Trace Context headers across synchronous and asynchronous boundaries
- **Centralized Logging:** Structured JSON logs aggregated in a log management platform (Splunk, ELK, or Anypoint Monitoring)
- **Metrics:** Error rate, average latency and 95th percentile (time taken by 95% of requests), circuit breaker state, retry rate, queue depth, message consumer lag — per API and per country
- **Alerts:** Alerts based on the allowed error margin of the service level agreement (SLO) that trigger on-call response

---

Integration Patterns Applied

Retry with Exponential Backoff and Jitter

Applied at the Process API layer for transient failures (network timeouts, 5xx responses from System APIs). Avoids simultaneous retry avalanches during backend recovery. Formula: `wait = min(cap, base * 2^attempt) + random_jitter`.

Circuit Breaker

Each System API connector is wrapped by a circuit breaker. After a configurable failure threshold, the circuit opens and returns a fast-fail response — protecting Salesforce and other backends from cascading overload. States: Closed → Open → Half-Open → Closed.

Idempotency

Every mutating operation (policy issuance, payment processing) requires an idempotency key in the request header. The Process API layer deduplicates requests within a configurable time window, preventing duplicate

records in Salesforce from retried requests.

Bulkhead

The bulkhead pattern applies resource isolation at two levels:

1. **Experience API layer:** Each channel has its own thread pool and rate limit. If the web portal receives a traffic spike or an attack, only its Experience API is affected — the mobile app and broker portal keep operating normally. No channel can exhaust another's resources.

2. **Process API / System layer:** MuleSoft thread pools are partitioned by country and by flow criticality. A flood of requests for Colombia does not exhaust the threads serving Venezuela. Critical flows (quotes, payments) are isolated from batch and reporting flows.

The result is that a failure or spike in any channel or country is contained — it does not propagate to the rest of the system.

Cache

- **Quote cache:** Product catalog and pricing cached at the Experience API layer (TTL: 5 minutes) — reduces Salesforce API calls for high-frequency quote requests
- **Token cache:** OAuth access tokens cached until expiry — eliminates per-request authentication overhead
- **Reference data cache:** Per-country configuration, product catalog, coverage rules — refreshed on schedule, not per request

Async Messaging

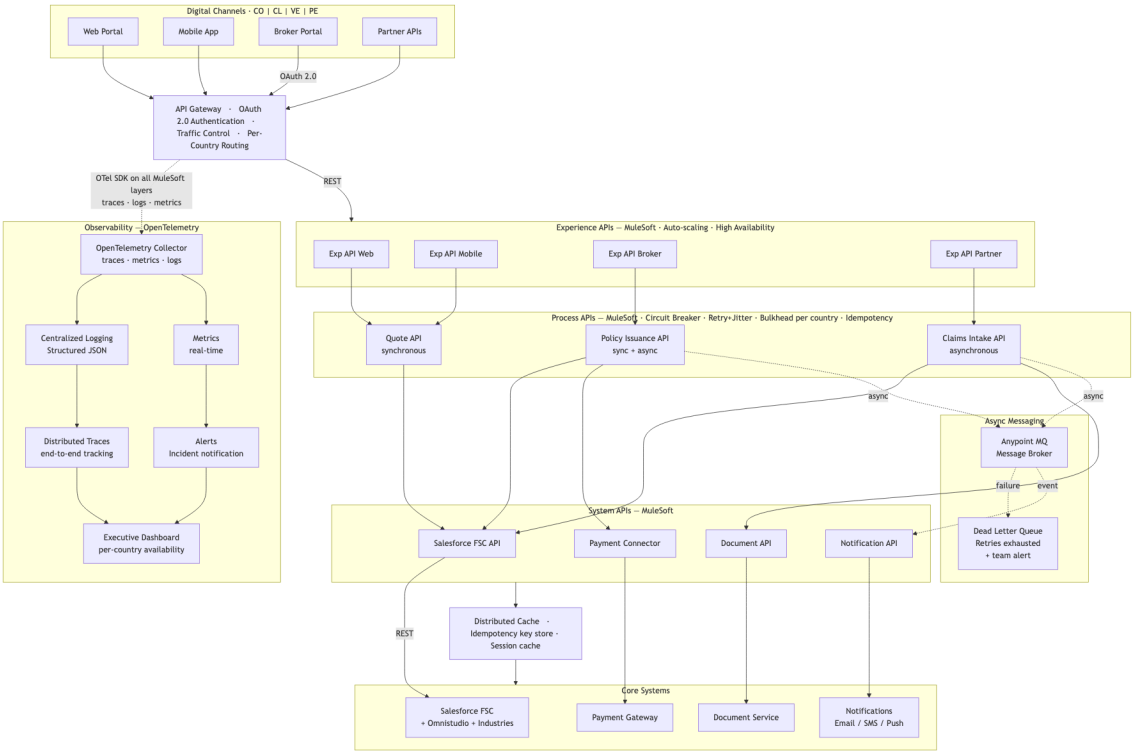
Policy issuance, claims notification, and cross-country data synchronization are routed to the async messaging layer (per ADR-002). Producers (Process APIs) publish events and return an acknowledgment immediately. Consumers process events independently with retry and DLQ support.

High Availability and Scalability

- MuleSoft CloudHub 2.0 (or Runtime Fabric on-premises): active-active deployment across multiple availability zones
- Auto-scaling rules based on CPU utilization and message queue depth
- Salesforce API call volume managed through bulkhead isolation — each country has a dedicated Salesforce Connected App with its own API limit allocation
- Load balancer with health checks at the Experience API layer — unhealthy workers removed from rotation without downtime
- HA at the database layer for the idempotency key store (distributed cache, e.g. Redis) with replication

---

2. Architecture Diagram



---

3. Technical Roadmap — 12 Weeks

Three parallel workstreams executed over 12 weeks:

Track 1 — Reliability

Week	Activity
1	Audit existing circuit breaker and retry configurations across all MuleSoft APIs
2	Define SLOs per API (availability, p95 latency, error rate)
3	Implement bulkhead thread pool isolation per country in MuleSoft
4	Deploy idempotency key validation for all mutating Process APIs
5	Implement retry with exponential backoff + jitter for all System API calls
6	Reliability review: controlled failure tests (chaos engineering) on circuit breaker scenarios
7	Implement cache layer: product catalog, OAuth tokens
8	Load testing: validate auto-scaling and bulkhead isolation under peak traffic
9	Implement Dead Letter Queue processing and alerting
10	Controlled failure tests: simulate Salesforce degradation; validate circuit breaker and fallback
11	Multi-country HA validation: simulate availability zone failure

12	Closure: all SLOs met, all critical paths tested

Track 2 — Integration Modernization

Week	Activity
1	Inventory all existing System APIs and identify duplication across countries
2	Establish API design standards and semantic versioning policy (e.g. v1.2.3)
3	Refactor per-country duplicate connectors into shared System APIs
4	Migrate the 3 highest-traffic integrations to the new System API pattern
5	Implement async messaging for the policy issuance flow
6	Implement async messaging for the claims notification flow
7	Publish reusable Experience API templates to Anypoint Exchange
8	Migrate second batch of per-country integrations to the shared System API pattern
9	Implement idempotency on the message consumer (async deduplication)
10	API governance review: deprecate legacy direct integrations
11	Complete Anypoint Exchange asset library: all reusable APIs documented
12	Closure: no direct Salesforce integrations bypassing MuleSoft

Track 3 — Observability and Operations

Week	Activity
1	Deploy OpenTelemetry collector; instrument the top 5 critical Process APIs
2	Implement structured JSON logging with correlation ID propagation
3	Build first metrics dashboard: error rate, latency, circuit breaker state
4	Define alert rules based on the allowed error margin of the service level agreement (SLO)
5	Instrument async flows: queue depth, message consumer lag, DLQ rate
6	First on-call runbook: open circuit breaker, DLQ spike, latency degradation
7	End-to-end distributed traces: trace propagation across sync and async boundaries
8	SLO review: adjust thresholds based on 4 weeks of real operational data
9	Capacity planning report: Salesforce API limits per country, MuleSoft worker utilization
10	Finalize on-call runbook for all P1/P2 scenarios
11	Executive observability report: SLO compliance, incident trends
12	Closure: traces, logs, metrics, alerts, and runbooks fully operational

Workstream Summary

Reliability — Weeks 1–12: Establish and enforce resilience patterns (circuit breaker, retry, bulkhead, idempotency, cache) across all integration flows. Validate under real failure conditions.

Integration Modernization — Weeks 1–12: Consolidate fragmented per-country integrations into a shared System API layer. Eliminate direct Salesforce access. Publish reusable assets to Anypoint Exchange. Adopt async messaging for long-running flows.

Observability and Operations — Weeks 1–12: Instrument all critical flows with OpenTelemetry. Build dashboards, define SLOs, configure alerts, and produce operational runbooks. Transition from reactive to proactive operations.

Section B – Reusable Integration Framework

Overview

The integration framework is a reusable Python module (`src/framework.py`) that provides a single entry point for all outbound HTTP calls from a service. Every integration goes through this client — resilience patterns are applied once here, without duplicating them at each integration point.

Components

1. Centralized Configuration (`IntegrationConfig`)

All tunable parameters are declared in a single dataclass. In production, these values are loaded from environment variables or a configuration service (MuleSoft properties, AWS Parameter Store, etc.).

```
config = IntegrationConfig(
    max_retries=3,
    base_backoff_seconds=0.5,
    max_backoff_seconds=10.0,
    jitter_factor=0.3,
    timeout_seconds=5.0,
    circuit_breaker_failure_threshold=5,
    circuit_breaker_recovery_seconds=30.0,
    service_name="policy-issuance-api"
)
```

Design decision: Centralizing configuration avoids magic numbers scattered across the code and allows runtime tuning without code changes.

---

2. Retry with Exponential Backoff and Jitter

Formula: `wait = min(cap, base × 2^attempt) + random_jitter`

Attempt	Base delay	With cap	+ Jitter (30%)	Approx. wait
0 (1st retry)	0.5s	0.5s	±0.15s	~0.5–0.65s

1 (2nd retry)	1.0s	1.0s	±0.30s	~1.0–1.30s
2 (3rd retry)	2.0s	2.0s	±0.60s	~2.0–2.60s

Why exponential backoff: Progressive delays give the downstream system increasing time to recover between attempts. A fixed retry interval can overload a partially recovering backend.

Why jitter: When multiple clients fail simultaneously (e.g., a brief network outage), they all retry at the same interval without jitter — creating a simultaneous retry avalanche that prevents the recovering backend from stabilizing. Jitter spreads retries over time, smoothing the load.

Why 3 retries: Based on observed transient failure windows — most transient errors resolve within 2 retries. A 4th retry adds latency without proportional benefit and may compromise the calling service's SLO.

---

3. Circuit Breaker

Three-state machine that protects downstream systems from overload during failure windows:

```
CLOSED ──(failure threshold)──> OPEN ──(recovery window)──> HALF-OPEN
   ^                                                          |
   └──────────────────(successful probe call)──────────────────┘
```

State	Behavior
Closed	Normal operation. Failures are counted.
Open	All calls are rejected immediately (fast fail). The downstream system rests.
Half-Open	One probe call is allowed. Success → Closed. Failure → Open.

Why the circuit breaker matters: Without it, retry storms from multiple clients hit a failing downstream system — preventing it from recovering. The circuit breaker breaks this cycle by stopping calls entirely during the recovery window.

Configuration: 5 failures → Open; 30s recovery window before Half-Open probe call.

---

4. Per-Request Timeout

Every HTTP call is bounded by a configurable timeout (default: 5 seconds). This prevents a slow downstream system from blocking a thread indefinitely — critical in user-facing synchronous flows where the total response time budget is bounded by the calling service's SLA.

Design decision: The timeout is always configured lower than the calling service's own timeout, to ensure the framework can handle the failure gracefully before the caller's deadline expires.

---

5. Idempotency Key Support

Mutating operations (POST, PUT, PATCH) accept an idempotency key. The framework stores successful responses indexed by that value. Duplicate requests return the cached response immediately without making a network call.

```
response = client.call(
    url="https://salesforce-api/policies",
```

```
    method="POST",
    body={"account_id": "001...", "product": "life"},
    idempotency_key="policy-issuance-req-abc123"
)
```

Why idempotency is essential for retry: Without it, a request that timed out but was already processed by the downstream system (e.g., Salesforce created the policy) would generate a duplicate record on retry. The idempotency key makes retries safe.

Production implementation: The store would be a distributed cache (Redis) shared across all MuleSoft workers, with a TTL matching the maximum expected retry window.

---

6. Unified Structured Logging

Each log entry is a JSON object with a consistent schema:

```
{
  "level": "warning",
  "event": "call_failed_retrying",
  "timestamp": "2026-03-28T19:00:00Z",
  "url": "https://salesforce-api/policies",
  "method": "POST",
  "attempt": 1,
  "error": "HTTP 503 from ...: Service Unavailable",
  "wait_seconds": 1.23,
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736"
}
```

Why structured logging: Free-text logs cannot be reliably analyzed or queried in log aggregation platforms (Splunk, ELK). A fixed JSON schema allows filtering by `trace_id`, `idempotency_key`, or `event` type across millions of entries — essential for incident diagnosis.

---

7. OpenTelemetry Trace Propagation

Each outbound request carries trace headers (`traceparent`, `tracestate`) following the W3C Trace Context standard — the universal format for propagating trace IDs between services. This allows the observability platform to join the full call chain — from the digital channel through MuleSoft Process APIs to Salesforce and back — even across async messaging boundaries.

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
tracestate:  sura=00f067aa0ba902b7
```

In production: Replace the lightweight `TraceContext` class with the `opentelemetry-sdk` package. The framework creates a record for each outbound call (span), captures status and latency, and exports them to the OpenTelemetry Collector (which feeds visualization tools like Jaeger or Datadog).

Usage Example

```python
from framework import IntegrationClient, IntegrationConfig, TraceContext

config = IntegrationConfig(
    max_retries=3,
    timeout_seconds=5.0,
    circuit_breaker_failure_threshold=5,
    service_name="policy-issuance-api"
)

client = IntegrationClient(config=config, circuit_name="salesforce-system-api")
trace = TraceContext()  # Root span for this request

response = client.call(
    url="https://salesforce-system-api/policies",
    method="POST",
    body={"account_id": "001ABC", "product_code": "LIFE_CO"},
    idempotency_key="issuance-2026-03-28-001ABC-LIFE",
    trace_context=trace
)
```

Design Decisions Summary

Decision	Rationale
Single entry point `IntegrationClient`	Resilience patterns applied once, not duplicated per integration
Exponential backoff with jitter	Avoids simultaneous retry avalanche; gives downstream system progressive recovery time
Circuit breaker separate from retry	Retry handles transient errors; circuit breaker handles sustained outages — distinct failure modes require distinct responses
Idempotency at framework level	Safe retries for all mutating operations without the caller managing it
Structured JSON logging	Machine-readable logs for reliable queries in aggregation platforms
W3C standard trace headers	Universal propagation format; compatible with all major observability platforms
In-memory idempotency store (demo)	Production equivalent: Redis with TTL; same interface, distributed backing

Section C – Demo Service and Reliability Testing

Overview

The demo service simulates a **Policy Issuance Process API** that calls a **Salesforce System API** using the Section B integration framework. The upstream system is intentionally unstable — it simulates the failure conditions a real insurance integration faces in production.

The demo requires no external dependencies and runs with a single command.

How to Run

Requirements: Python 3.8+

```
# From the repository root
python src/demo.py
```

No additional packages needed. The unstable upstream server runs in the same process on port 9999.

What the Demo Shows

Architecture

```
Demo Service (Policy Issuance API)
    │
    ├── IntegrationClient (framework.py)
    │       ├── Idempotency check
    │       ├── Circuit breaker check
    │       ├── HTTP call with timeout (5s)
    │       ├── Retry + exponential backoff + jitter
    │       └── Structured logging + trace propagation
    │
    └──> Unstable Upstream (simulated Salesforce System API :9999)
             ├── 30% success (HTTP 200)
             ├── 60% transient failure (HTTP 503)
             └── 10% timeout (response after 8s > client timeout 5s)
```

Failure Scenarios Demonstrated

Scenario 1 — Retry under Transient Failures

The upstream returns HTTP 503 randomly. The framework retries up to 3 times with exponential backoff + jitter before giving up.

Example console output:

```
{"level": "warning", "event": "call_failed_retrying",
 "url": "http://localhost:9999/v1/policies",
 "attempt": 0, "error": "HTTP 503 from ...", "wait_seconds": 0.52, "trace_id":
"4bf92f..."}

{"level": "warning", "event": "call_failed_retrying",
 "url": "http://localhost:9999/v1/policies",
```

```
 "attempt": 1, "error": "HTTP 503 from ...", "wait_seconds": 1.18, "trace_id":
"4bf92f..."}

{"level": "info", "event": "call_succeeded",
 "url": "http://localhost:9999/v1/policies", "attempt": 2, "trace_id": "4bf92f..."}
```

How resilience mitigates the problem: The client retries automatically. The calling service receives a successful response without knowing about the underlying failures. Progressive backoff gives the upstream time to recover between attempts.

Scenario 2 — Per-Request Timeout

The upstream sleeps for 8 seconds. The client applies a 5-second timeout, fails fast, and retries.

How resilience mitigates the problem: Without a timeout, the client thread would be blocked for 8+ seconds — violating the calling service's SLA (e.g., a web user waiting for a quote). The timeout forces a fast fail, allowing the retry to potentially reach a healthy upstream worker.

Scenario 3 — Circuit Breaker Opens

After 3 consecutive failures (demo threshold), the circuit opens. Subsequent calls are rejected immediately without touching the upstream.

Example console output:

```
{"level": "warning", "event": "circuit_breaker_opened",
 "circuit": "salesforce-system-api",
 "failure_count": 3, "recovery_seconds": 10.0}

{"level": "warning", "event": "circuit_breaker_rejected",
 "url": "http://localhost:9999/v1/policies",
 "circuit": "salesforce-system-api", "trace_id": "7c3a1f..."}
```

How resilience mitigates the problem: The circuit breaker stops hitting the already-failing upstream, giving it space to recover. The caller receives a fast, predictable `CircuitBreakerOpenError` — instead of waiting through multiple timeouts and retries.

Scenario 4 — Idempotency Cache Hit

A request with a previously successful idempotency key is resubmitted. The framework returns the cached response without making any network call.

Example console output:

```
{"level": "info", "event": "idempotency_cache_hit",
 "idempotency_key": "demo-policy-001", "trace_id": "4bf92f..."}
```

How resilience mitigates the problem: In production, a mobile client might retry a policy issuance request after a network drop. Without idempotency, Salesforce would create a duplicate policy. The idempotency key guarantees the operation executes exactly once — even across multiple retries.

Observability Output

Each event produces a structured JSON log entry visible in the console. In production, these entries flow to the centralized log aggregation platform. The `trace_id` field links all log entries from the same request chain — across retries, services, and synchronous/asynchronous boundaries.

Production Equivalents

Demo Component	Production Equivalent
In-process unstable HTTP server	Salesforce System API under degradation
In-memory idempotency store	Redis distributed cache with TTL
Trace context with `print()`	OpenTelemetry SDK → Collector → Jaeger/Datadog
Single Python process	MuleSoft CloudHub worker with multiple replicas
`CircuitBreakerOpenError`	MuleSoft error handler returning HTTP 503 + Retry-After header

Section D – Technical Decision Record

ADR-001: Centralized Integration Platform vs. Decentralized Team-Owned Integrations

Date: 2026-03-28 **Status:** Accepted **Decision makers:** Technical Lead, Architecture Team

---

Context

Sura operates a multi-country Digital Direct Channel where every inbound and outbound request flows through a centralized middleware layer (MuleSoft Anypoint Platform) that mediates between Salesforce (central insurance system of record) and external systems, partners, and digital channels.

As the organization scales across countries and product lines, teams face increasing pressure to deliver integrations faster. Two competing models emerge:

- **Option A:** Maintain and strengthen the centralized MuleSoft integration platform, managed by a dedicated integration team.
- **Option B:** Allow product teams to own and build their own integrations directly, reducing dependency on a central team.

This decision directly impacts delivery speed, governance, operational resilience, and long-term maintainability.

Options Considered

Option A — Centralized Integration Platform (MuleSoft API-Led)

All integrations are built, deployed, and operated through MuleSoft Anypoint Platform following a three-layer API-led architecture:

- **Experience APIs:** Channel-specific adapters (mobile, web, broker portals)
- **Process APIs:** Orchestration and business logic
- **System APIs:** Connectors to Salesforce, core insurance systems, and external providers

A dedicated integration team owns the platform, enforces standards, and publishes reusable assets to Anypoint Exchange.

Advantages:

- Single compliance point for security, rate limiting, and observability
- Reusable API assets reduce duplication across countries
- Consistent logging, tracing, and error handling across all integrations
- Simpler compliance and auditability (insurance sector regulatory requirements)
- Resilience patterns (retry, circuit breaker, idempotency) applied once, inherited everywhere

Disadvantages:

- Central team becomes a bottleneck for high-velocity product teams
- Requires strong API design governance and versioning discipline
- Higher upfront investment in platform expertise and tooling

---

Option B — Decentralized Team-Owned Integrations

Each product team builds and operates its own integrations using the technology of their choice. Teams are responsible for their own reliability, security, and observability.

Advantages:

- Teams move faster without waiting for a central team
- Technology flexibility per team context
- Lower organizational dependency

Disadvantages:

- Fragmented observability — no unified view of system health
- Security and compliance gaps between teams of different maturity
- Duplication of integration logic across countries and channels
- Resilience patterns implemented inconsistently or not at all
- High operational cost: N teams operating N integration stacks
- In a regulated industry (insurance), decentralized ownership significantly increases audit complexity

---

Decision

We recommend Option A — Centralized Integration Platform, with specific tactical adjustments to reduce bottlenecks.

Rationale:

1. **The regulatory context demands governance.** Insurance operations across multiple countries require consistent audit trails, data sovereignty controls, and security compliance. A centralized platform provides a single compliance perimeter.
2. **Resilience at scale requires consistency.** Patterns such as circuit breakers, idempotency, and retry with exponential backoff must be applied uniformly. Decentralized ownership produces inconsistent resilience — some teams implement it, others don't.

3. **Reuse reduces total cost.** Salesforce System APIs and core insurance Process APIs, built once, can be reused across all countries and channels. Decentralization rebuilds them per team.
4. **The bottleneck is solvable without decentralizing.** The real pain is team velocity, not the centralized model itself. The solution is an **internal open contribution model**: product teams contribute to the integration platform through governed self-service patterns, published templates, and reusable connectors — while the central team focuses on platform reliability and governance, not attending tickets from each team.

Consequences

Positive:

- Unified observability across all integration flows
- Consistent resilience and security posture
- Reduced duplication of Salesforce and core system connectors
- Simpler compliance reporting across countries

Negative / Mitigations:

- The central team must act as a support team, not a control team — providing templates, accelerators, and self-service patterns to product teams
- Platform governance must be lightweight — heavy review processes defeat the purpose
- Requires investment in MuleSoft expertise and Anypoint Exchange asset library

---

ADR-002: Event-Driven vs. Synchronous Request-Response for Critical Flows

Date: 2026-03-28 **Status:** Accepted **Decision makers:** Technical Lead, Architecture Team

---

Context

The Digital Direct Channel handles multiple flow types with distinct characteristics:

- **Quote generation:** The user requests an insurance quote in real time — expects a response in under 3 seconds
- **Policy issuance:** Triggers downstream processes in Salesforce, payment gateways, and document generation
- **Claims notification:** Initiates a multi-step workflow across internal teams and external adjusters
- **Cross-country data synchronization:** Policy and customer data must remain consistent across per-country Salesforce instances

The question is: which flows should be **synchronous (request-response)** and which **event-driven (async messaging)**?

Options Considered

Option A — Synchronous Request-Response for All Flows

All operations are handled through synchronous HTTP calls via MuleSoft. The caller waits for a complete response before continuing.

Advantages:

- Simple programming model — easier to reason about and debug
- Immediate error feedback to the caller
- No additional messaging infrastructure required

Disadvantages:

- Tight coupling between systems — if the downstream fails, the entire flow fails
- Cannot handle high-volume spikes gracefully — load propagates directly to backends
- Long-running operations (claims, document generation) block the caller's thread
- Cascading failures: a slow Salesforce response degrades the entire channel

---

Option B — Event-Driven for Long-Running and High-Volume Flows, Synchronous for Real-Time Responses

A hybrid model where the communication pattern is selected based on the flow's characteristics:

Flow Type	Pattern	Rationale
Quote generation	Synchronous	User waits for the response; must be < 3s
Policy issuance	Event-driven (async)	Multi-step workflow; decouples channel from backend processing
Claims notification	Event-driven (async)	Initiates a long-running multi-team workflow
Cross-country data sync	Event-driven (async)	Eventual consistency is acceptable; volume can be high
Authentication / session	Synchronous	Security-sensitive; requires immediate validation

Advantages:

- Decouples producers from consumers — the channel remains responsive even if Salesforce is slow
- Absorbs traffic spikes through message queue buffering
- Enables retry and dead letter queue patterns for failed events
- Long-running workflows do not block user-facing threads

Disadvantages:

- Introduces eventual consistency — the data synchronizes with a small delay, not in real time; requires idempotency and deduplication
- More complex operational model — requires queue depth and message consumer lag monitoring
- Harder to trace the full end-to-end flow without proper correlation IDs and distributed traces

---

Decision

We recommend Option B — Hybrid model: event-driven for long-running and high-volume flows, synchronous for real-time user-facing responses.

Rationale:

1. **User experience dictates synchronous flows.** Quote generation and authentication cannot be asynchronous — users expect immediate feedback. Forcing them through a queue adds latency

without benefit.

2. **Backend resilience requires decoupling.** Policy issuance and claims notification trigger complex downstream workflows in Salesforce and external systems. A synchronous chain means any slow or failing downstream degrades the entire channel. An event-driven approach isolates failures to the consuming service.
3. **Queues absorb traffic spikes, not backends.** In a multi-country digital channel, peak traffic (campaign launches, renewal periods) can saturate synchronous backends. Message queues act as buffers, smoothing the load without scaling backends beyond what is necessary.
4. **Idempotency makes async safe.** Each event carries an idempotency key, enabling safe replay without duplicate processing — a requirement already present in the integration framework (Section B).

Consequences

Positive:

- The channel remains responsive under backend degradation
- Long-running workflows are reliable and retriable
- Traffic spikes do not cascade into backend failures
- Each flow type uses the communication pattern best suited to its characteristics

Negative / Mitigations:

- Operational complexity increases — mitigated by centralized observability (Section A) with queue depth and message consumer lag dashboards
- Eventual consistency requires idempotency discipline — applied at the integration framework level
- End-to-end tracing requires correlation IDs propagated across sync and async boundaries — applied through OpenTelemetry trace propagation (Section B)